

SQL 4 : Modification d'une BD

Guillaume Guex guillaume.guex@unil.ch

Création de tables dérivées

Nous avons vu comment créer un schéma de table grâce à **CREATE TABLE**, puis comment la remplir manuellement avec **INSERT INTO ... VALUES**.

Nous allons voir ici comment **créer des tables à partir de données existantes** déjà dans la base de données. Ces opérations sont fondamentales pour peupler une base de données.

Dans ce cours, nous allons utiliser ici la base de donnée golf (fichier `golf_dump.sql`).

Peuplement d'une table existante

Après la création d'une table avec `CREATE TABLE`, le `INSERT INTO` peut également s'utiliser avec un `SELECT`. Commençons par créer un schéma d'une table.

```
CREATE TABLE phonebook  
(  
    entry_id INT AUTO_INCREMENT NOT NULL,  
    first_name VARCHAR(100),  
    last_name VARCHAR(100),  
    phone_number INT,  
    coach_name VARCHAR(100),  
    PRIMARY KEY (entry_id)  
);
```

Notez que nous avons désigné `entry_id` comme `AUTO_INCREMENT` et que le type pour `phone_number` est différent.

Création directe

Une fois la table créée, nous pouvons la **peupler** à partir de `INSERT INTO ... SELECT`.

```
INSERT INTO phonebook (first_name, last_name, phone_number, coach_name)
SELECT m1.first_name, m1.last_name, m1.phone, m2.last_name
FROM member m1
LEFT JOIN member m2 ON m1.coach = m2.member_id;
```

	entry_id	first_name	last_name	phone_number	coach_name
1	1	Melissa	McKenzie	963,270	Nolan
2	2	Michael	Stone	983,223	[NULL]
3	3	Brenda	Nolan	442,649	[NULL]
4	4	Helen	Branch	589,419	[NULL]
5	5	Sarah	Beck	226,596	[NULL]
6	6	Sandra	Burton	244,493	Nolan
7	7	William	Cooper	722,954	Nolan
8	8	Thomas	Spence	697,720	[NULL]
9	9	Barbara	Olson	370,186	[NULL]
10	10	Robert	Pollard	617,681	Cooper
11	11	Thomas	Sexton	268,936	Cooper
12	12	Daniel	Wilcox	665,393	[NULL]
13	13	Thomas	Schmidt	867,492	Nolan
14	14	Deborah	Bridges	279,087	Cooper
15	15	Betty	Young	507,813	[NULL]
16	16	Jane	Gilmore	459,558	Nolan
17	17	William	Taylor	137,353	Cooper
18	18	Robert	Reed	994,664	Cooper
19	19	Carolyn	Willis	688,378	[NULL]
20	20	Susan	Kent	707,217	[NULL]

La colonne `entry_id` s'est remplie automatiquement (attention lors d'un re-remplissage, le compteur ne se remet pas à zéro).

On peut également créer et peupler directement une table avec `CREATE TABLE ... AS SELECT`.

```
CREATE TABLE teamA AS
SELECT t.team_name, m.last_name, m.first_name
FROM team t JOIN member m ON t.team_name = m.team
WHERE t.team_name = 'TeamA';
```

	A-Z team_name ▼	A-Z last_name ▼	A-Z first_name ▼
1	TeamA	Wilcox	Daniel
2	TeamA	Gilmore	Jane
3	TeamA	Taylor	William
4	TeamA	Reed	Robert

En faisant ainsi, les types des données sont dérivés des tables initiales.

Modification du schéma

La **modification du schéma** permet, entre-autre, de :

- Ajouter et supprimer des attributs.
- Ajouter ou supprimer des clés.
- Changer le type et les contraintes sur les attributs.

La commande commence par **ALTER TABLE**, puis une commande particulière qui va faire ces différentes opérations.

Ajouter ou supprimer un attribut

Pour **ajouter un attribut**, on utilise **ALTER TABLE ... ADD**.

tournament	
123 tournament_id	smallint(6) NOT NULL
A-Z tournament_name	char(20)
A-Z tournament_type	char(20)

```
ALTER TABLE tournament  
ADD tournament_point INT;
```

tournament	
123 tournament_id	smallint(6) NOT NULL
A-Z tournament_name	char(20)
A-Z tournament_type	char(20)
123 tournament_point	int(11)

On peut aussi facilement **supprimer un attribut**.

```
ALTER TABLE tournament  
DROP tournament_point;
```

Ajouter et supprimer une clé primaire

La commande `ALTER TABLE ... ADD` peut aussi servir à **ajouter une clé primaire**.

```
ALTER TABLE teamA  
ADD PRIMARY KEY last_name;
```

Et pour la **supprimer**, sans supprimer la colonne.

```
ALTER TABLE teamA  
DROP PRIMARY KEY;
```

On peut également créer **une nouvelle colonne directement comme clé** (`AUTO_INCREMENT` nécessaire).

```
ALTER TABLE teamA  
ADD teammate_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY;
```

	A-z team_name ▾	A-z last_name ▾	A-z first_name ▾	123 teammate_id ▾
1	TeamA	Gilmore	Jane	1
2	TeamA	Reed	Robert	2
3	TeamA	Taylor	William	3
4	TeamA	Wilcox	Daniel	4

Ajouter et supprimer une clé étrangère

Concernant les **clés étrangères**, il faut d'abord trouver son **nom de contrainte**.

	Name	Column	Owner	Ref Table	Type	Ref Object	On Delete	On Update
Columns								
Constraints	FK_Coach	—	member	member	FOREIGN KEY	PRIMARY	Restrict	Restrict
Foreign Keys	member_ibfk_1	—	member	member_type	FOREIGN KEY	PRIMARY	Restrict	Restrict
References	FK_Team	—	member	team	FOREIGN KEY	PRIMARY	Restrict	Restrict
Triggers								

On peut donc **supprimer** la référence à member_type avec :

```
ALTER TABLE member  
DROP FOREIGN KEY member_ibfk_1;
```

Ajouter une clé étrangère suit une syntaxe similaire à celle qu'on utilise lors de la création :

```
ALTER TABLE member  
ADD FOREIGN KEY member_type REFERENCES member_type(member_type);
```

Changer le type et les contraintes

Finalement, `ALTER TABLE ... MODIFY` nous permet de **modifier le type et les contraintes des attributs**.

```
ALTER TABLE member  
MODIFY phone INT UNSIGNED;
```

Si la conversion est possible, SQL la fait automatiquement. Si ce n'est pas le cas, une erreur est générée.

```
ALTER TABLE member  
MODIFY first_name INT;  
-- ERREUR
```

D'autres opérations sont encore possibles avec `ALTER TABLE`, pour en savoir plus <https://mariadb.com/kb/en/alter-table/>.

Modification des données

On peut également **modifier les données** sans toucher au schéma de la base de données. Cela consiste en :

- Modifier des données sur certains attributs à certaines lignes.
- Supprimer certaines lignes.
- Supprimer entièrement le contenu de tables.

Notez que la modification des données prendra de l'ampleur lorsque nous verrons les **fonctions**.

Modification d'une entrée

Pour modifier **une seule entrée**, on utilise **UPDATE ... SET** avec une condition (un **WHERE**) pour sélectionner la ligne en question.

```
UPDATE member
SET last_name = 'Sexton',
    first_name = 'Tom'
WHERE member_id = 290;
```

Notez que cela va générer des erreurs si les types ne sont pas compatibles.

```
UPDATE member
SET joindate = 'Very early'
WHERE member_id = 290;
-- ERREUR
```

Modification de plusieurs entrées

La **modification de plusieurs entrées** se fait de manière similaire. On va d'abord modifier le type des données pour accepter les nouvelles entrées.

```
ALTER TABLE member  
MODIFY gender VARCHAR(10);
```

Puis on modifie les données.

```
UPDATE member  
SET gender = 'Female'  
WHERE gender = 'F';
```

```
UPDATE member  
SET gender = 'Male'  
WHERE gender = 'M';
```

La **suppression de plusieurs entrées** prend la syntaxe suivante :

```
DELETE FROM member  
WHERE member_type = 'Social';
```

Notez qu'il y aura un message d'erreur si l'une des entrées est référencée ailleurs. Il faut parfois supprimer des clés étrangères au préalable.

Pour vider intégralement une table, on fait :

```
TRUNCATE phonebook;
```

Son schéma reste cependant disponible (**DROP TABLE** l'efface entièrement).